

Musical Instrument Builder · Full Curriculum

Single session (2.5 hrs) · 8-12 yrs · Beginner → Intermediate

Full facilitator curriculum **PREMIUM**

Overview

Build a playable electronic instrument on a breadboard.

DETAIL

Track	Arduino Robotics & Electronics
Format	Single session (2.5 hrs)
Ages	8-12 yrs
Level	Beginner → Intermediate
Price	from KES 5,000
Standalone	No
Prerequisites	Buzzer melody (free, recommended)

Course Overview

Musical Instrument Builder is a single, fast-paced 2.5-hour workshop in which each learner assembles one continuously-growing breadboard circuit and, by the end of the session, walks away with a fully playable, multi-mode electronic instrument. Rather than three separate builds, the three parts are three *upgrades* to the same circuit: what starts as a simple hand-controlled ultrasonic theremin becomes an eight-key button piano, and finally a tempo-adjustable recorder/looper that can play back a short melody.

The teaching arc deliberately layers concepts: Part 1 introduces sensors, analog-to-frequency mapping and the `tone()` function through the emotionally exciting theremin. Part 2 introduces digital input scanning, arrays and musical frequency tables through a hands-on piano. Part 3 ties everything together with a potentiometer, mode-switching logic and simple data recording (arrays of notes

and timings), giving learners a taste of how real loop-pedals and synth sequencers work.

By the close of the session every learner keeps their instrument circuit on its breadboard — wired, coded and demoed — plus the printed note-frequency chart and their own annotated code, so they can keep experimenting with it at home.

Learning Outcomes

- Wire and calibrate an HC-SR04 ultrasonic distance sensor and use `pulseIn()` to measure distance.
- Use `map()` to convert a sensor reading into a musical frequency and drive a piezo buzzer with `tone()` / `noTone()`.
- Wire multiple tactile pushbuttons using `INPUT_PULLUP` and scan them in a loop using arrays.
- Read an analog potentiometer and map its value onto a useful range (tempo/speed).
- Understand and implement simple state-machine logic (mode switching) with debounced pushbuttons.
- Record a short sequence of events (notes + timing) into arrays and play it back — the basic idea behind digital sequencers.
- Debug a growing circuit systematically (isolate a fault to sensor, wiring or code).

Materials Master List

ITEM	SPEC	QTY PER STUDENT/ GROUP	NOTES
Microcontroller board	Arduino Uno R3 (or compatible clone)	1	Pixel Electronics/Nerokas stock these; must have USB-B port
USB cable	USB-A to USB-B, 1 m	1	One per board
Breadboard	830-point solderless breadboard	1	Full size needed for 10 buttons + sensor + pot
Ultrasonic sensor	HC-SR04 (4-pin: VCC, TRIG, ECHO, GND)	1	Mount on small cardboard/ lolly-stick stand for stability

Tactile pushbuttons	12 mm 4-leg momentary pushbutton	10	8 for piano keys, 1 for mode, 1 for record/play
Potentiometer	10 kΩ linear rotary potentiometer, breadboard-friendly (3-pin)	1	Tempo control
Buzzer	Passive piezo buzzer, 2-pin, 5 V	1	Must be <i>passive</i> (works with <code>tone()</code>), not the self-oscillating active type
LED	5 mm LED, any colour	1	Mode/record indicator
Resistor	220 Ω, 1/4 W	1	In series with the LED
Jumper wires	Male-to-male, assorted lengths	~40	Breadboard wiring
Labels	Masking tape strip + marker	1 set	To label the 8 note buttons C-C
Handout	Printed note-frequency chart (C4-C5) and pin map	1 per student	Provided in this pack — print in advance
Laptop	Arduino IDE 1.8.x or 2.x installed, correct board/driver (CH340 driver if using a clone)	1 per student/pair	Pre-install before session to save time
Small stand/base (optional)	Cardboard strip or 3D-printed clip	1	Holds HC-SR04 upright for the theremin

Session Plans

Part 1 — Ultrasonic Theremin

Objectives

- Explain how an ultrasonic sensor measures distance using sound echo timing.
- Wire the HC-SR04 and a piezo buzzer to the Arduino correctly.
- Use `map()` to turn a distance reading into a musical pitch and control it live with hand movement.

Timing (40 minutes)

- **0-5 min:** Hook — facilitator plays a real theremin sound clip (or demos a pre-built unit), asks "how could a sensor make sound follow your hand?" Introduces the idea of sending a sound pulse and timing its echo.
- **5-15 min:** Guided wiring — sensor and buzzer onto the breadboard, following the wiring list below. Facilitator checks each board before power-on.

- **15-25 min:** Upload the Part 1 sketch together, line by line, pausing at `pulseIn()` and `map()` to explain what's happening. Fix upload/driver issues here.
- **25-35 min:** Free play — students wave a hand over the sensor and hear pitch change; facilitator challenges them to change the `map()` range (e.g. swap 2000/100 to 3000/50) and observe the effect.
- **35-40 min:** Quick check-for-understanding questions, then explain that this circuit will grow, not be dismantled, for Part 2.

Step-by-step

1. Show the HC-SR04 up close: point out TRIG (sends the pulse) and ECHO (listens for it bouncing back).
2. Draw on the board: sound out → hits hand → bounces back → time taken tells us distance.
3. Direct students to place the sensor at one end of the breadboard, buzzer at the other end, leaving the middle rows free (Parts 2 and 3 will fill them).
4. Wire strictly following the table below — insist students dry-fit before powering on.
5. Open Arduino IDE, select correct Board and Port (Tools menu), paste in the Part 1 sketch.
6. Upload. If no sound: check buzzer polarity is irrelevant (passive buzzers have no polarity) but check pin A1 wiring and open Serial Monitor at 9600 baud to see if distance values are printing sensibly (2-50 range).
7. Let students hold a hand 5 cm, 20 cm, 40 cm away and describe what they hear.
8. Challenge: ask them to predict what happens if they swap the two numbers in `map(cm, 2, 50, 2000, 100)` to `map(cm, 2, 50, 100, 2000)` — then test it.

Build detail

Materials for this part: Arduino Uno, breadboard, HC-SR04, passive piezo buzzer, 6-8 jumper wires.

Wiring/assembly (component → pin):

- HC-SR04 VCC → Arduino 5V
- HC-SR04 GND → Arduino GND
- HC-SR04 TRIG → Arduino D2
- HC-SR04 ECHO → Arduino D3

- Buzzer leg 1 → Arduino A1
- Buzzer leg 2 → Arduino GND

```
// PART 1 – ULTRASONIC THEREMIN
// Hand distance from the sensor controls the pitch of the buzzer.

#define TRIG_PIN 2      // sends the ultrasonic pulse
#define ECHO_PIN 3      // listens for the echo
#define BUZZER_PIN A1   // passive piezo buzzer

void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  Serial.begin(9600);    // so we can watch the distance readings
}

void loop() {
  long duration, cm;

  // Send a short 10-microsecond pulse out of TRIG
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // Measure how long the echo took to return (30ms timeout = ~5m max range)
  duration = pulseIn(ECHO_PIN, HIGH, 30000);

  // Convert time to centimetres (speed of sound math, divided by 2 for round-trip)
  cm = duration / 29 / 2;

  Serial.print("Distance: ");
  Serial.print(cm);
  Serial.println(" cm");

  // Only make sound within a sensible playing range
  if (cm > 2 && cm < 50) {
    int freq = map(cm, 2, 50, 2000, 100); // closer hand = higher pitch
    tone(BUZZER_PIN, freq);
  } else {
    noTone(BUZZER_PIN);
  }

  delay(50); // small pause so readings are stable
}
```

Checks for understanding

- What two things does the HC-SR04 sensor need to do to measure distance?
- If we moved our hand closer to the sensor, would the pitch in our code get higher or lower? Why?
- What does `noTone()` do, and why do we need it here?

Common mistakes & fixes

MISTAKE	FIX
No sound at all	Check buzzer is on A1 and GND, check Serial Monitor is printing distances — if not, recheck TRIG/ECHO wiring
Distance reads 0 or huge random numbers	Loose jumper on ECHO or TRIG; reseal wires firmly into breadboard holes
Sound is constant no matter hand position	Hand is outside the 2–50 cm range in the code — move closer/further, or widen the range in <code>map()</code>
Board won't upload	Wrong Board/Port selected in Tools menu, or missing CH340 driver on a clone board

Part 2 — Button Piano

Objectives

- Wire eight tactile buttons as digital inputs using internal pull-up resistors.
- Use an array to store the frequencies of a full musical octave (C4–C5).
- Scan multiple buttons in a loop and play the correct note for whichever is pressed.

Timing (45 minutes)

- **0–5 min:** Explain octaves and note frequencies using the printed chart; show that each key must map to one specific frequency number.
- **5–20 min:** Wire all 8 buttons into the middle of the breadboard, label each with masking tape (C, D, E, F, G, A, B, C).
- **20–30 min:** Walk through the Part 2 sketch together, focussing on the two arrays (`notePins` , `noteFreqs`) and the `for` loop that scans them.
- **30–40 min:** Practice time — students try to play "Twinkle Twinkle" or "Happy Birthday" opening notes using the chart.
- **40–45 min:** Checks for understanding + preview that Part 3 will let them record what they just played.

Step-by-step

1. Hand out the note-frequency chart; ask students to identify which note is "middle C" (C4).

2. Explain `INPUT_PULLUP`: the pin reads HIGH normally, and drops to LOW only when the button is pressed and connects the pin to GND — so buttons only need one wire to GND, not a resistor.
3. Direct wiring of all 8 buttons, one leg to a digital pin D4–D11, the other leg to the breadboard's GND rail.
4. Label buttons left to right with tape: C D E F G A B C.
5. Upload Part 2 sketch. Test each button one at a time, checking Serial Monitor prints the correct note number.
6. Explain the `break;` in the loop — it means only the first pressed button found gets played (avoids two notes clashing).
7. Let students play simple tunes from the chart; circulate to help with sticky buttons or wiring.

Build detail

Materials for this part (in addition to Part 1): 8x tactile pushbuttons, 8–10 jumper wires, masking tape labels.

Wiring/assembly (component → pin):

- Button 1 (C4) → D4, other leg → GND
- Button 2 (D4) → D5, other leg → GND
- Button 3 (E4) → D6, other leg → GND
- Button 4 (F4) → D7, other leg → GND
- Button 5 (G4) → D8, other leg → GND
- Button 6 (A4) → D9, other leg → GND
- Button 7 (B4) → D10, other leg → GND
- Button 8 (C5) → D11, other leg → GND
- Buzzer stays on A1/GND from Part 1 (sensor wiring stays in place but is unused this part)

```
// PART 2 – BUTTON PIANO
// Eight tactile buttons mapped to a full octave (C4 to C5).

#define BUZZER_PIN A1

const int notePins[8] = {4, 5, 6, 7, 8, 9, 10, 11}; // one pin per key
const int noteFreqs[8] = {262, 294, 330, 349, 392, 440, 494, 523}; // C4 D4 E4 F4 G4 A4 B4 C5

void setup() {
```

```

pinMode(BUZZER_PIN, OUTPUT);
for (int i = 0; i < 8; i++) {
  pinMode(notePins[i], INPUT_PULLUP); // no resistor needed, pin reads LOW when pressed
}
Serial.begin(9600);
}

void loop() {
  bool pressed = false;

  for (int i = 0; i < 8; i++) {
    if (digitalRead(notePins[i]) == LOW) { // button pressed = connects pin to GND
      tone(BUZZER_PIN, noteFreqs[i]);
      Serial.print("Playing note: ");
      Serial.println(i + 1);
      pressed = true;
      break; // only play the first key found this loop
    }
  }

  if (!pressed) {
    noTone(BUZZER_PIN); // silence when nothing is pressed
  }
}

```

Checks for understanding

- Why does the button reading go LOW, not HIGH, when pressed?
- What would happen to the code if we removed the `break;` line and two keys were pressed at once?
- Which array stores the pin numbers, and which stores the pitches?

Common mistakes & fixes

MISTAKE	FIX
A button does nothing	Check both legs — one must reach the GND rail, the other the correct digital pin
Button always plays even when not pressed	Pin not set to <code>INPUT_PULLUP</code> , or wire is shorting to GND permanently — check for stray wire touching the rail
Wrong note plays for a key	Pin number in wiring doesn't match its position in <code>notePins[]</code> — check against the pin map
Notes sound "stuck on"	Buzzer wiring loose causing false LOW reads; reseal jumpers

Part 3 — Tempo & Recording

Objectives

- Wire a potentiometer and two extra buttons (mode + record/play) into the existing circuit.
- Understand and use a simple mode-switching state machine.
- Record a short melody into arrays and play it back at an adjustable tempo.

Timing (45 minutes)

- **0-5 min:** Introduce the idea of a "looper pedal" — record now, play back later, control the speed with a knob.
- **5-15 min:** Wire the potentiometer, mode button, record button and LED into remaining breadboard rows.
- **15-30 min:** Upload the full integrated sketch. Walk through the three modes and how the mode button cycles between them (LED blinks confirm the mode).
- **30-40 min:** Students record a short 4-8 note melody in Record mode, then play it back while turning the potentiometer to speed up/slow down.
- **40-45 min:** Transition straight into showcase — no dismantling needed, this is the final instrument.

Step-by-step

1. Explain the 3 modes: Mode 0 Theremin (from Part 1), Mode 1 Piano (from Part 2), Mode 2 Record & Play (new).
2. Wire the potentiometer: middle pin to A0, outer pins to 5V and GND (either way round is fine — turning direction just reverses).
3. Wire the MODE button to D12→GND and the REC/PLAY button to A2→GND, both using `INPUT_PULLUP` like the piano keys.
4. Wire the LED: long leg through the 220Ω resistor to D13, short leg to GND.
5. Upload the full sketch. Open Serial Monitor to see mode names print as the MODE button is pressed.
6. Demonstrate: press MODE once → 1 LED blink (Theremin); again → 2 blinks (Piano); again → 3 blinks (Record & Play).
7. In Record & Play mode: press REC/PLAY once — LED goes solid, they play a few notes on the keys, press REC/PLAY again — LED goes off and the melody plays back automatically.

8. Turn the potentiometer and repeat the record/playback to hear the tempo change on playback.
9. Have each student save/rename their sketch file with their name for the showcase.

Build detail

Materials for this part (in addition to Parts 1 & 2): 10 kΩ potentiometer, 2x extra tactile pushbuttons (mode, record/play), 1x LED, 1x 220Ω resistor, 6–8 jumper wires.

Wiring/assembly (component → pin):

- Potentiometer wiper (middle pin) → A0
- Potentiometer outer pin 1 → 5V
- Potentiometer outer pin 2 → GND
- MODE button → D12, other leg → GND
- REC/PLAY button → A2, other leg → GND
- LED anode (long leg) → 220Ω resistor → D13
- LED cathode (short leg) → GND
- All Part 1 and Part 2 wiring remains unchanged

```
// PART 3 – FULL INSTRUMENT: THEREMIN + PIANO + TEMPO/RECORDING
// Mode button cycles between 3 modes. Record & Play mode captures
// a short melody and plays it back at a speed set by the potentiometer.

#define TRIG_PIN    2
#define ECHO_PIN    3
#define BUZZER_PIN  A1
#define MODE_BUTTON 12
#define REC_BUTTON  A2
#define POT_PIN     A0
#define LED_PIN     13

const int notePins[8] = {4, 5, 6, 7, 8, 9, 10, 11};
const int noteFreqs[8] = {262, 294, 330, 349, 392, 440, 494, 523}; // C4 to C5

int mode = 0; // 0 = Theremin | 1 = Piano | 2 = Record & Play

bool lastModeState = HIGH;
bool lastRecState  = HIGH;
unsigned long lastModePress = 0;
unsigned long lastRecPress  = 0;
const unsigned long DEBOUNCE = 250; // ms

bool isRecording = false;
const int MAX_NOTES = 20;
```

```

int recNote[MAX_NOTES];
unsigned long recGap[MAX_NOTES]; // ms since the previous note
int noteCount = 0;
unsigned long lastEventTime = 0;

void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(MODE_BUTTON, INPUT_PULLUP);
  pinMode(REC_BUTTON, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
  for (int i = 0; i < 8; i++) pinMode(notePins[i], INPUT_PULLUP);

  Serial.begin(9600);
  Serial.println("Instrument ready - Mode 0: THEREMIN");
  announceMode();
}

void loop() {
  checkModeButton();

  if (mode == 0) {
    runTheremin();
    digitalWrite(LED_PIN, LOW);
  } else if (mode == 1) {
    runPiano();
    digitalWrite(LED_PIN, LOW);
  } else if (mode == 2) {
    runRecordAndPlay();
  }
}

// ----- MODE SWITCHING -----
void checkModeButton() {
  bool state = digitalRead(MODE_BUTTON);
  if (state == LOW && lastModeState == HIGH && (millis() - lastModePress) > DEBOUNCE) {
    mode = (mode + 1) % 3; // cycle 0 -> 1 -> 2 -> 0
    lastModePress = millis();
    noTone(BUZZER_PIN);
    announceMode();
  }
  lastModeState = state;
}

void announceMode() {
  const char* names[3] = {"THEREMIN", "PIANO", "RECORD & PLAY"};
  Serial.print("Mode -> ");
  Serial.println(names[mode]);
  // Blink the LED (mode+1) times so we can see the mode without the laptop
  for (int i = 0; i <= mode; i++) {
    digitalWrite(LED_PIN, HIGH);
    delay(150);
    digitalWrite(LED_PIN, LOW);
    delay(150);
  }
}

// ----- MODE 0: THEREMIN -----
void runTheremin() {

```

```

    long duration, cm;
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    duration = pulseIn(ECHO_PIN, HIGH, 30000);
    cm = duration / 29 / 2;

    if (cm > 2 && cm < 50) {
        int freq = map(cm, 2, 50, 2000, 100);
        tone(BUZZER_PIN, freq);
    } else {
        noTone(BUZZER_PIN);
    }
}

// ----- MODE 1: PIANO -----
void runPiano() {
    bool anyPressed = false;
    for (int i = 0; i < 8; i++) {
        if (digitalRead(notePins[i]) == LOW) {
            tone(BUZZER_PIN, noteFreqs[i]);
            anyPressed = true;
            break;
        }
    }
    if (!anyPressed) noTone(BUZZER_PIN);
}

// ----- MODE 2: RECORD & PLAY -----
void runRecordAndPlay() {
    digitalWrite(LED_PIN, isRecording ? HIGH : LOW); // solid LED while recording

    // Toggle recording on/off with the REC/PLAY button
    bool recState = digitalRead(REC_BUTTON);
    if (recState == LOW && lastRecState == HIGH && (millis() - lastRecPress) > DEBOUNCE) {
        lastRecPress = millis();
        if (!isRecording) {
            isRecording = true;
            noteCount = 0;
            lastEventTime = millis();
            Serial.println("Recording started...");
        } else {
            isRecording = false;
            Serial.println("Recording stopped. Playing back...");
            playback();
        }
    }
    lastRecState = recState;

    if (isRecording) {
        // Capture notes as they are played on the 8 keys
        for (int i = 0; i < 8; i++) {
            if (digitalRead(notePins[i]) == LOW && noteCount < MAX_NOTES) {
                tone(BUZZER_PIN, noteFreqs[i]);
                recNote[noteCount] = i;
                recGap[noteCount] = millis() - lastEventTime;
                lastEventTime = millis();
            }
        }
    }
}

```

```

        noteCount++;
        delay(250);          // simple debounce: one press = one recorded note
        noTone(BUZZER_PIN);
    }
}
} else {
    // Not recording - keys can still be played live for practice
    runPiano();
}
}

void playback() {
    int potValue = analogRead(POT_PIN); // 0-1023
    // Map pot to a speed multiplier from 0.5x (slow) to 2.0x (fast)
    float tempoScale = map(potValue, 0, 1023, 50, 200) / 100.0;

    for (int i = 0; i < noteCount; i++) {
        delay(recGap[i] * tempoScale);
        tone(BUZZER_PIN, noteFreqs[recNote[i]], 200);
        delay(200);
    }
    noTone(BUZZER_PIN);
    Serial.println("Playback finished.");
}

```

Checks for understanding

- What does the LED do differently in Record & Play mode compared to the other two modes?
- If we turn the potentiometer to make `tempoScale` smaller, will playback be faster or slower? Why?
- What two pieces of information does the code store for every recorded note?

Common mistakes & fixes

MISTAKE	FIX
Mode button skips modes / cycles too fast	Debounce too short or button legs miswired — check DEBOUNCE value and wiring, press deliberately once
Recording doesn't play anything back	Check REC/PLAY was pressed twice (start, then stop) and that notes were actually pressed while LED was solid
Potentiometer has no effect on tempo	Wiper (middle pin) not on A0, or outer pins swapped with GND/5V reversed (usually still works, just reversed direction)
Melody plays but sounds robotic/ too fast always	Notes recorded too close together (fast pressing) — encourage clearer gaps between key presses when recording

Assessment & Showcase

Assessment is continuous and practical rather than written. During each part, the facilitator circulates and checks off three things per student: (1) the wiring matches the pin map without shorts, (2) the sketch uploads and runs without errors, and (3) the student can explain in their own words what the code is doing (e.g. "the closer my hand is, the higher the pitch because `map()` flips the numbers"). Use the Checks for Understanding questions above as informal spot-checks — a student who can answer at least one confidently per part is on track.

"Done" for the final take-home instrument means: the student can demonstrate all three modes on request (cycle with the MODE button), play a recognisable few notes in Piano mode, wave a hand for a clear pitch sweep in Theremin mode, and record + play back at least a 4-note melody with an audible tempo change when the potentiometer is turned.

For the showcase (last 10–15 minutes if time allows, or as the natural end of Part 3): each student gets 60–90 seconds on a small "stage" (a table with their breadboard) to perform. Suggested format — 20 seconds Theremin freestyle, 20 seconds Piano tune, then a full record-and-playback demonstration. Facilitator or peers can call out a tempo request ("play it back twice as fast!") for the student to adjust live using the potentiometer. Take a photo of each finished breadboard for the programme's portfolio before students carefully pack up their kit to take home.

Extension & Home Challenges

- **Easy:** Add a second LED that flickers in time with the notes being played (turn it on briefly inside `runPiano()` whenever a key is pressed) so the instrument has a light show, not just sound.
- **Medium:** Extend `noteFreqs[]` and add 4 more buttons (or reuse the mode/rec buttons temporarily) to cover a second octave, C4 to C6, and update the printed chart to match.
- **Hard:** Modify the Record & Play code to store *two* melodies in separate arrays (e.g. using the mode button held for over 1 second to switch between "Melody A" and "Melody B" while recording), then play them back one after another to create a simple two-part song.